LIST OF EXPERIMENTS

S.NO	DATE	NAME OF THE EXPERIMENT	PAGE NO	SIGNATURE
1		Implementation of Informed search algorithms (A*, memory-bounded A*)		
2		Implement naïve Bayes models		
3		Implement Bayesian Networks		
4		Build Regression models i) Linear Regression ii) Multiple Linear Regression		
5		Implement Decision Tree and Random forests		
6		Build SVM models		
7		Implement ensembling techniques i) Averaging Method ii) AdaBoost iii) Gradient Descent		
8		Implement clustering algorithms i) K-Means Clustering ii) Hierarchical Clustering iii) DBSCAN Clustering		

EXPT NO: 1 DATE:	**Implementation of Informed search algorithms (A*, memory-bounded A*)**

Aim:

To write a code to implementation of Informed search algorithm (A*, memory-bounded A*) using Python

Algorithm:

Step 1: Initialize the open set with the start node and the closed set as empty.
Step 2: Initialize g(start_node) to 0 and parents(start_node) to start_node.
Step 3: While open set is not empty, do the following:
  a. Select the node with the lowest f() value from the open set.
  b. If this node is the stop node or it has no neighbors, exit the loop.
  c. For each neighbor of the current node:
    i. If it is not in open set or closed set, add it to the open set, set its parent to the current node, and calculate its g value.
    ii. If it is in the open set and its g value is greater than the current node's g value plus the weight of the edge between them, update its g value and parent node.
    iii. If it is in the closed set and its g value is greater than the current node's g value plus the weight of the edge between them, remove it from the closed set and add it to the open set with the updated g value and parent node.
  d. If no node is selected, there is no path between start and stop nodes.
  e. If the stop node is selected, construct the path from start to stop using the parent nodes and return the path.
  f. Remove the selected node from the open set and add it to the closed set.
Step 4: If the open set becomes empty, there is no path between start and stop nodes.

Program

```
def aStarAlgo(start_node, stop_node):
    open_set = set(start_node)
    closed_set = set()
    g = {}              #store distance from starting node
    parents = {}        # parents contains an adjacency map of all nodes
    #distance of starting node from itself is zero
    g[start_node] = 0
    #start_node is root node i.e it has no parent nodes
    #so start_node is set to its own parent node
    parents[start_node] = start_node
    while len(open_set) > 0:
        n = None
        #node with lowest f() is found
        for v in open_set:
            if n == None or g[v] + heuristic(v) < g[n] + heuristic(n):
                n = v
        if n == stop_node or Graph_nodes[n] == None:
            pass
        else:
            for (m, weight) in get_neighbors(n):
```

```
            #nodes 'm' not in first and last set are added to first
            #n is set its parent
            if m not in open_set and m not in closed_set:
                open_set.add(m)
                parents[m] = n
                g[m] = g[n] + weight
            #for each node m,compare its distance from start i.e g(m) to the
            #from start through n node
            else:
                if g[m] > g[n] + weight:
                    #update g(m)
                    g[m] = g[n] + weight
                    #change parent of m to n
                    parents[m] = n
                    #if m in closed set,remove and add to open
                    if m in closed_set:
                        closed_set.remove(m)
                        open_set.add(m)
        if n == None:
            print('Path does not exist!')
            return None

        # if the current node is the stop_node
        # then we begin reconstructin the path from it to the start_node
        if n == stop_node:
            path = []
            while parents[n] != n:
                path.append(n)
                n = parents[n]
            path.append(start_node)
            path.reverse()
            print('Path found: {}'.format(path))
            return path
        # remove n from the open_list, and add it to closed_list
        # because all of his neighbors were inspected
        open_set.remove(n)
        closed_set.add(n)
    print('Path does not exist!')
    return None
#define fuction to return neighbor and its distance
#from the passed node
def get_neighbors(v):
    if v in Graph_nodes:
        return Graph_nodes[v]
    else:
        return None
```

```
#for simplicity we we'll consider heuristic distances given
#and this function returns heuristic distance for all nodes
def heuristic(n):
    H_dist = {
        'A': 11,
        'B': 6,
        'C': 5,
        'D': 7,
        'E': 3,
        'F': 6,
        'G': 5,
        'H': 3,
        'I': 1,
        'J': 0
    }
    return H_dist[n]
#Describe your graph here
Graph_nodes = {
    'A': [('B', 6), ('F', 3)],
    'B': [('A', 6), ('C', 3), ('D', 2)],
    'C': [('B', 3), ('D', 1), ('E', 5)],
    'D': [('B', 2), ('C', 1), ('E', 8)],
    'E': [('C', 5), ('D', 8), ('I', 5), ('J', 5)],
    'F': [('A', 3), ('G', 1), ('H', 7)],
    'G': [('F', 1), ('I', 3)],
    'H': [('F', 7), ('I', 2)],
    'I': [('E', 5), ('G', 3), ('H', 2), ('J', 3)],
}
aStarAlgo('A', 'J')
```

EXPECTED OUTPUT

```
Path found: ['A', 'F', 'G', 'I', 'J']
['A', 'F', 'G', 'I', 'J']
```

Result :

The above code was executed and verified successfully.

EXPT NO: 2 DATE:	**Implement Naïve Bayes Models**

Aim:

To write a code to implement Naive Bayes Model using Python

Algorithm:

Step 1: Import the required libraries:

1. pandas for data handling
2. sklearn for splitting data and applying classification algorithms
3. CountVectorizer for vectorizing text data
4. MultinomialNB for Naive Bayes algorithm
5. metrics for evaluating the model

Step 2: Read the dataset in a pandas dataframe.
Step 3: Rename the columns to appropriate names.
Step 4: Remove any unnecessary columns from the dataframe.
Step 5: Convert the class labels to numeric values (0 for ham and 1 for spam) using lambda function.
Step 6: Split the dataset into training and testing sets using the train_test_split function from sklearn.
Step 7: Create a CountVectorizer object to convert text data into vectors.
Step 8: Use the fit_transform method of the CountVectorizer object to transform the training set into feature vectors.
Step 9: Use the transform method of the CountVectorizer object to transform the testing set into feature vectors.
Step 10: Create a MultinomialNB object for Naive Bayes classification.
Step 11: Use the fit method of the MultinomialNB object to train the model on the training data.
Step 12: Use the predict method of the MultinomialNB object to predict the class labels of the testing set.
Step 13: Use the metrics from sklearn to evaluate the performance of the model:

1. Use accuracy_score to calculate the accuracy of the model.
2. Use confusion_matrix to calculate the confusion matrix of the model.
3. Use classification_report to generate a report on the precision, recall, and f1-score of the model.

Program

Snippet Code 1

```
import pandas as pd
df = pd.read_csv('spam.csv',encoding='cp1252')
df
```

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN

2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN
...	...	...	...	...	...
5567	spam	This is the 2nd time we have tried 2 contact u...	NaN	NaN	NaN
5568	ham	Will Ì_ b going to esplanade fr home?	NaN	NaN	NaN
5569	ham	Pity, * was in mood for that. So...any other s...	NaN	NaN	NaN
5570	ham	The guy did some bitching but I acted like i'd...	NaN	NaN	NaN
5571	ham	Rofl. Its true to its name	NaN	NaN	NaN

5572 rows × 5 columns

```
df.rename(columns={"v1":"class_label","v2":"message"},inplace=True)
df
```

	class_label	message	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN

3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN
...	...	...	...	...	...
5567	spam	This is the 2nd time we have tried 2 contact u...	NaN	NaN	NaN
5568	ham	Will Ì_ b going to esplanade fr home?	NaN	NaN	NaN
5569	ham	Pity, * was in mood for that. So...any other s...	NaN	NaN	NaN
5570	ham	The guy did some bitching but I acted like i'd...	NaN	NaN	NaN
5571	ham	Rofl. Its true to its name	NaN	NaN	NaN

5572 rows × 5 columns

```
df.drop(['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'],axis=1,inplace=True)
df
```

	class_label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

...	...	...
5567	spam	This is the 2nd time we have tried 2 contact u...
5568	ham	Will Ì_ b going to esplanade fr home?
5569	ham	Pity, * was in mood for that. So...any other s...
5570	ham	The guy did some bitching but I acted like i'd...
5571	ham	Rofl. Its true to its name

5572 rows × 2 columns

```
df.class_label.value_counts()
```

Output

```
ham     4825
spam     747
Name: class_label, dtype: int64
```

```
# convert class lable from string to numeric

df["class_label"]=df["class_label"].apply(lambda x: 1 if x == "spam" else 0)

df
```

	class_label	message
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...
...	...	...

5567	1	This is the 2nd time we have tried 2 contact u...
5568	0	Will Ì_ b going to esplanade fr home?
5569	0	Pity, * was in mood for that. So...any other s...
5570	0	The guy did some bitching but I acted like i'd...
5571	0	Rofl. Its true to its name

5572 rows × 2 columns

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
                                df['message'],
                                df['class_label'],
                                test_size = 0.3,
                                random_state = 0)
```

```
y_train.value_counts()
```

Output

```
0    3391
1     509
Name: class_label, dtype: int64
```

```
y_test.value_counts()
```

Output

```
0    1434
1     238
Name: class_label, dtype: int64
```

```
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer(
                lowercase=True,      # convert to lowercase before tokenizing
                stop_words='english' # remove stop words
                )
```

```
x_train_transformed = vectorizer.fit_transform(x_train) #gives  vector for x_train

x_test_transformed  = vectorizer.transform(x_test) #gives  vector for x_test

x_train_transformed
```

Output

```
<3900x6840 sparse matrix of type '<class 'numpy.int64'>'
	with 30364 stored elements in Compressed Sparse Row format>
```

```
x_train_transformed.toarray()
```

Output

```
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]], dtype=int64)
```

```
from sklearn.naive_bayes import MultinomialNB

classifier = MultinomialNB()

classifier.fit(x_train_transformed, y_train)
```

Output

```
MultinomialNB()
```

```
ytest_predicted_labels = classifier.predict(x_test_transformed)

ytest_predicted_labels
```

Output

```
array([0, 0, 0, ..., 0, 0, 0], dtype=int64)
```

```
y_test.count()
```

Output

```
1672
```

```
y_test  # actual labels in test dataset
ytest_predicted_labels # predicted labels for test dataset
```

Output

```
4456   0
690    0
944    0
3768   0
1189   0
      ..
4833   0
```

```
3006   0
509    0
1761   0
1525   0
Name: class_label, Length: 1672, dtype: int64
```

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
print ('Accuracy Score :',accuracy_score(y_test, ytest_predicted_labels))
```

Output

```
Accuracy Score : 0.9850478468899522
```

```
results = confusion_matrix(y_test, ytest_predicted_labels)
print(results)
```

Output

```
[[1423   11]
 [  14  224]]
```

```
print (classification_report(y_test, ytest_predicted_labels) )
```

Output

```
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      1434
           1       0.95      0.94      0.95       238
    accuracy                           0.99      1672
   macro avg       0.97      0.97      0.97      1672
weighted avg       0.98      0.99      0.99      1672
```

Result :

The above code was executed and verified successfully.

EXPT NO: 3 DATE:	Implement Bayesian Networks

Aim:

To write a code to implement Bayesian Network using Python

Algorithm:

Step 1: Import required libraries such as numpy, pandas, csv, MaximumLikelihoodEstimator, BayesianModel, and VariableElimination.
Step 2: Read the heart.csv file using pandas and replace missing values '?' with NaN.
Step 3: Display sample instances from the dataset and the attributes and datatypes of the dataset.
Step 4: Define a BayesianModel object with nodes and edges using the BayesianModel function.
Step 5: Use the MaximumLikelihoodEstimator to learn Conditional Probability Distributions (CPD) from the dataset using the fit function of the model object.
Step 6: Use the VariableElimination class to perform inference with the Bayesian network using the query function.
1.Step 7: Print the probability of HeartDisease given evidence of restecg.

Program

```
import numpy as np

import pandas as pd

import csv

from pgmpy.estimators import MaximumLikelihoodEstimator

from pgmpy.models import BayesianModel

from pgmpy.inference import VariableElimination

heartDisease = pd.read_csv('heart.csv')

heartDisease = heartDisease.replace('?',np.nan)

print('Sample instances from the dataset are given below')

print(heartDisease.head())

print('\n Attributes and datatypes')

print(heartDisease.dtypes)

model=
BayesianModel([('age','heartdisease'),('sex','heartdisease'),('exang','heartdisease'),('cp','he
artdisease'),('heartdisease','restecg'),('heartdisease','chol')])
```

```
print('\nLearning CPD using Maximum likelihood estimators')

model.fit(heartDisease,estimator=MaximumLikelihoodEstimator)

print('\n Inferencing with Bayesian Network:')

HeartDiseasetest_infer = VariableElimination(model)

print('\n 1. Probability of HeartDisease given evidence= restecg')

q1=HeartDiseasetest_infer.query(variables=['heartdisease'],evidence={'restecg':1})

print(q1)
```

EXPECTED OUTPUT

```
Learning CPD using Maximum likelihood estimators

 Inferencing with Bayesian Network:

 1. Probability of HeartDisease given evidence= restecg

   +-----------------+---------------------+
   | heartdisease    |   phi(heartdisease) |
   +=================+=====================+
   | heartdisease(0) |              0.1012 |
   +-----------------+---------------------+
   | heartdisease(1) |              0.0000 |
   +-----------------+---------------------+
   | heartdisease(2) |              0.2392 |
   +-----------------+---------------------+
   | heartdisease(3) |              0.2015 |
   +-----------------+---------------------+
   | heartdisease(4) |              0.4581 |
   +-----------------+---------------------+
```

```
2. Probability of HeartDisease given evidence= cp

+-----------------+---------------------+
| heartdisease    |   phi(heartdisease) |
+=================+=====================+
| heartdisease(0) |              0.3610 |
+-----------------+---------------------+
| heartdisease(1) |              0.2159 |
+-----------------+---------------------+
| heartdisease(2) |              0.1373 |
+-----------------+---------------------+
| heartdisease(3) |              0.1537 |
+-----------------+---------------------+
| heartdisease(4) |              0.1321 |
+-----------------+---------------------+
```

Result :

The above code was executed and verified successfully.

EXPT NO: 4.i. DATE:	**Build Regression models (i) Linear Regression**

Aim:

To write a program for implementing Linear regression model using Python

Algorithm:

Step 1: Import the required libraries:

1. pandas for data handling
2. numpy for mathematical processing
3. matplotlib for data visualization
4. sklearn for splitting data and applying classification algorithms.

Step 2: Read the dataset in a pandas dataframe.
Step 3: Rename the columns to appropriate names.
Step 4: Remove any unnecessary columns from the dataframe.
Step 5: Split the dataset into training and testing sets using the train_test_split function from sklearn.
Step 6: Create LinearRegression object for regression model.
Step 7: Use fit method of LinearRegression object to train the model on the training data.
Step 8: Use the predict method of the LinearRegression object to predict the class labels of the testing set.
Step 9: Visualize the result using scatter plot of matplotlib.

Program

```
# importing the dataset
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
dataset = pd.read_csv('D:\\KIOT\\AAAA\\AIML_LAB\\Salary_Data.csv')
dataset.head()
# data preprocessing
X = dataset.iloc[:, :-1].values  #independent variable array
y = dataset.iloc[:,1].values  #dependent variable vector
# splitting the dataset
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=1/3,random_state=0)
# fitting the regression model
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train,y_train) #actually produces the linear eqn for the data
# predicting the test set results
y_pred = regressor.predict(X_test)
y_pred
y_test
# visualizing the results
#plot for the TRAIN
plt.scatter(X_train, y_train, color='red') # plotting the observation line
```

```
plt.plot(X_train, regressor.predict(X_train), color='blue') # plotting the regression line
plt.title("Salary vs Experience (Training set)") # stating the title of the graph
plt.xlabel("Years of experience") # adding the name of x-axis
plt.ylabel("Salaries") # adding the name of y-axis
plt.show() # specifies end of graph
#plot for the TEST
plt.scatter(X_test, y_test, color='red')
plt.plot(X_train, regressor.predict(X_train), color='blue') # plotting the regression line
plt.title("Salary vs Experience (Testing set)")
plt.xlabel("Years of experience")
plt.ylabel("Salaries")
plt.show()
```

EXPECTED OUTPUT

Sample dataset

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0

Output

Result :

The above code was executed and verified successfully.

EXPT NO: 4.ii. DATE:	**Build Regression models (ii) Multiple Linear Regression**

Aim:

To write a program for implementing Multiple Linear regression model using Python

Algorithm:

Step 1: Import the required libraries:
pandas for data handling
numpy for mathematical processing
matplotlib for data visualization
sklearn for splitting data and applying classification algorithms.
LabelEncoder and OneHotEncoder from sklearn.preprocessing.
Step 2: Read the dataset in a pandas dataframe.
Step 3: Rename the columns to appropriate names.
Step 4: Remove any unnecessary columns from the dataframe.
Step 5: Split the datasets into features and target using iloc method.
Step 6: Encode the categorical feature in the feature set using LabelEncoder and OneHotEncoder methods.
Step 7: Split the dataset into training and testing sets using the train_test_split function from sklearn.
Step 8: Use the predict method of the LinearRegression object to predict the class labels of the testing set.
Step 9: Print the actual target values and predicted target values using the y test and ypred variables.

Program

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
dataset = pd.read_csv('D:\\KIOT\AAAA\AIML_LAB\\50_Startups.csv')
dataset.head()
# data preprocessing
X = dataset.iloc[:,:-1].values
y = dataset.iloc[:,4].values
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
labelEncoder_X = LabelEncoder()
X[:,3] = labelEncoder_X.fit_transform(X[ : , 3])
from sklearn.compose import ColumnTransformer
ct = ColumnTransformer([('encoder', OneHotEncoder(), [3])], remainder='passthrough')
X = np.array(ct.fit_transform(X), dtype=np.float)
X = X[:, 1:]
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
# Fitting the model
```

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
# predicting the test set results
y_pred = regressor.predict(X_test)
y_test
y_pred
```

EXPECTED OUTPUT

Sample data set:

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

Output

```
array([103015.20159795, 132582.27760817, 132447.73845176,  71976.09851257,
       178537.48221058, 116161.24230165,  67851.69209675,  98791.73374686,
       113969.43533013, 167921.06569553])
```

Result :

The above code was executed and verified successfully.

EXPT NO: 5.i. DATE:	**Build decision trees**

Aim:

To write a program for implementing Decision Tree model using Python

Algorithm:

Step 1: Import the required libraries:

1.pandas for data handling
2.sklearn for splitting data and applying classification algorithms.
3.DecisionTree classifier from sklearn.tree
4.plot_tree from sklearn.tree for plotting the decision tree.

Step 2: Read the dataset in a pandas dataframe.
Step 3: Remove any unnecessary columns from the dataframe.
Step 4: Split the dataset into training and testing sets using the train_test_split function from sklearn.
Step 5: Create DecisionTreeClassifier object to create decision tree classifier with Gini impurity.
Step 6: Use fit method to train the model on the training data.
Step 7: Visualize the result using plot_tree method.

Program

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
# Create the input data
data = {
   'income': [20, 30, 40, 50, 60, 70, 80, 90, 100, 110],
   'credit_score': [50, 60, 70, 80, 90, 100, 110, 120, 130, 140],
   'eligible': [0, 0, 0, 1, 1, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)
df.head()
```

Sample Dataset

	income	credit_score	eligible
0	20	50	0
1	30	60	0
2	40	70	0
3	50	80	1
4	60	90	1

```
# Split the data into features and target
X = df.drop(['eligible'], axis=1)
y = df['eligible']
# Create a decision tree classifier with Gini impurity
clf = DecisionTreeClassifier(criterion='gini')
```

```
# Fit the classifier to the data
clf.fit(X, y)
```

Output

```
DecisionTreeClassifier()

# Plot the decision tree

plot_tree(clf, feature_names=X.columns, class_names=['Not eligible', 'Eligible'], filled=True)
```

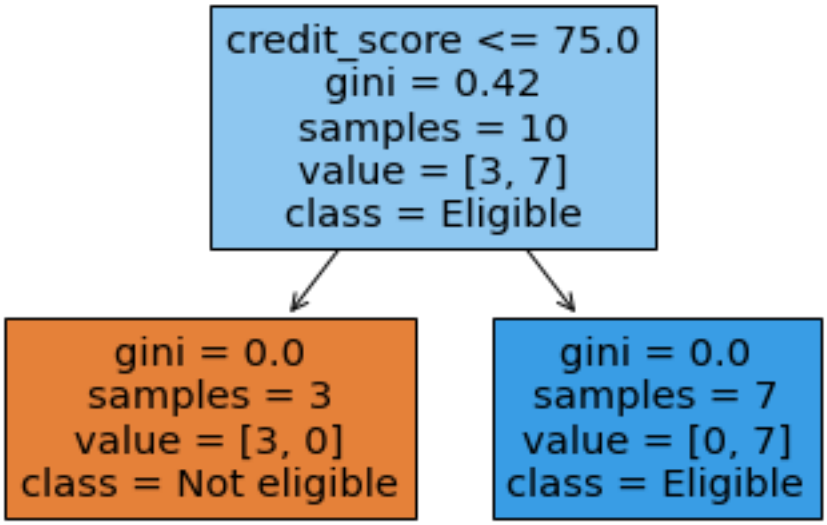
Output

```
Text(0.5, 0.75, 'credit_score <= 75.0\ngini = 0.42\nsamples = 10\nvalue = [3, 7]\nclass = Eligible'),

 Text(0.25, 0.25, 'gini = 0.0\nsamples = 3\nvalue = [3, 0]\nclass = Not eligible'),

 Text(0.75, 0.25, 'gini = 0.0\nsamples = 7\nvalue = [0, 7]\nclass = Eligible')]
```

EXPECTED OUTPUT



Result :

The above code was executed and verified successfully.

EXPT NO: 5.ii. DATE:	**Build Random forests**

Aim:

To write a program for implementing Random Forest model using Python

Algorithm:

Step 1: Import the required libraries:

1. pandas for data handling
2. numpy for mathematical operation
3. sklearn for splitting data and applying classification algorithms.
4. RandomForestRegressor from sklearn.ensemble
5. matplotlib for visualizing the result.
6. mean_squared_error from metrics.

Step 2: import load_boston dataset from sklearn.

Step 3: Split the dataset into training and testing sets using the train_test_split function from sklearn.

Step 4: Create RandomForestRegressor object.

Step 5: Use fit method to train the model on the training data and predict the results.

Step 6: Calculate the mean squared error of predicted values using mean_squared_error method.

Step 7: Visualize the result using barh plot.

Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_boston
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
# Load the Boston dataset
boston = load_boston()
X = boston.data
y = boston.target
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Create a Random Forest Regressor with 100 trees
rfr = RandomForestRegressor(n_estimators=100, random_state=42)
# Fit the model to the training data
rfr.fit(X_train, y_train)
```

Output

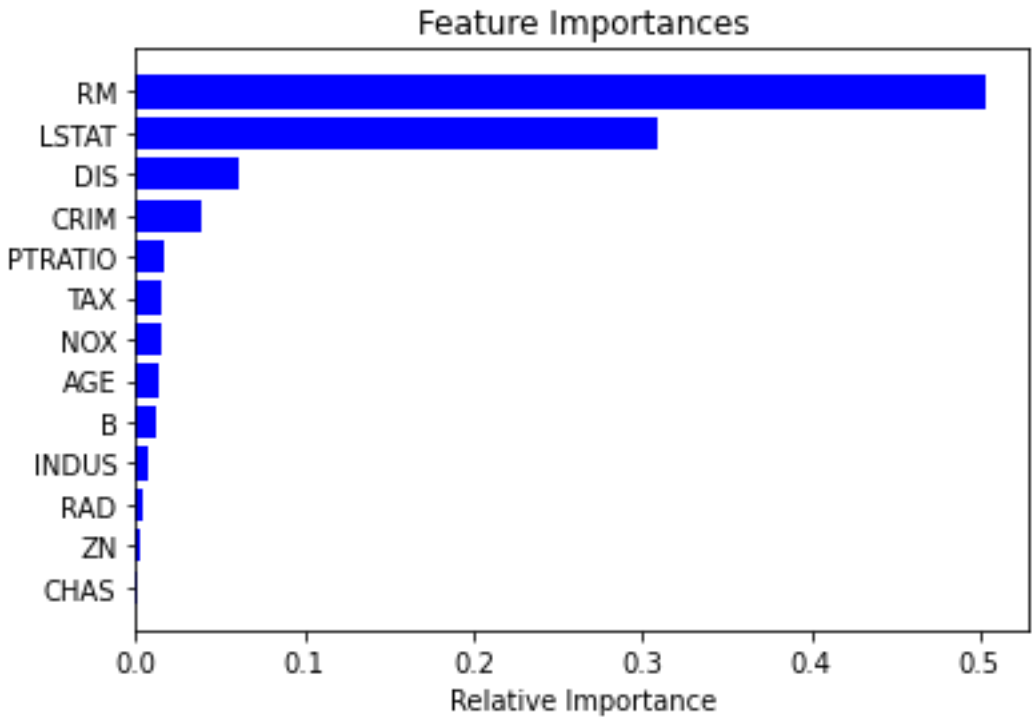
```
RandomForestRegressor(random_state=42)
# Predict the target values of the testing data
y_pred = rfr.predict(X_test)
# Calculate the mean squared error
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

Output

Mean Squared Error: 7.901513892156864

```
# Plot the feature importances
features = boston.feature_names
importances = rfr.feature_importances_
indices = np.argsort(importances)
plt.title('Feature Importances')
plt.barh(range(len(indices)), importances[indices], color='b', align='center')
plt.yticks(range(len(indices)), [features[i] for i in indices])
plt.xlabel('Relative Importance')
plt.show()
```

EXPECTED OUTPUT



Result :

The above code was executed and verified successfully.

EXPT NO: 6 DATE:	**Build SVM models**

Aim:

To write a program to implement the SVM model using Python

Algorithm:

Step 1: Import necessary libraries – pandas, numpy, matplotlib.pyplot, and sklearn.
Step 2: Load the dataset using pandas read_csv function and store it in the variable 'data'.
Step 3: Split the dataset into training and test samples using train_test_split function from sklearn. Store them in the variables 'training_set' and 'test_set'.
Step 4: Classify the predictors and target. Extract the first two columns as predictors and the last column as the target variable. Store them in the variables 'X_train' and 'Y_train' for the training set and 'X_test' and 'Y_test' for the test set.
Step 5: Encode the target variable using LabelEncoder from sklearn. Store it in the variable 'le'.
Step 6: Initialize the Support Vector Machine (SVM) classifier using SVC from sklearn with kernel type 'rbf' and random_state as 1.
Step 7: Fit the training data into the classifier using the fit() function.
Step 8: Predict the classes for the test set using the predict() function and store them in the variable 'Y_pred'.
Step 9: Attach the predictions to the test set for comparing using the code "test_set['Predictions'] = Y_pred".
Step 10: Calculate the accuracy of the predictions using confusion_matrix from sklearn.
Step 11: Visualize the classifier using matplotlib. Use the ListedColormap to color the graph and show the legend using the scatter plot.

Program

```
#Importing the dataset
import pandas as pd
data = pd.read_csv("D:\\KIOT\\AAAA\\AIML_LAB\\apples_and_oranges.csv")
#Splitting the dataset into training and test samples
from sklearn.model_selection import train_test_split
training_set, test_set = train_test_split(data, test_size = 0.2, random_state = 1)
#Classifying the predictors and target
X_train = training_set.iloc[:,0:2].values
Y_train = training_set.iloc[:,2].values
X_test = test_set.iloc[:,0:2].values
Y_test = test_set.iloc[:,2].values
#Initializing Support Vector Machine and fitting the training data
from sklearn.svm import SVC
classifier = SVC(kernel='rbf', random_state = 1)
classifier.fit(X_train,Y_train)
#Predicting the classes for test set
Y_pred = classifier.predict(X_test)
#Attaching the predictions to test set for comparing
test_set["Predictions"] = Y_pred
```

```
#Calculating the accuracy of the predictions
#We will calculate the accuracy using the confusion matrix as follows :
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_test,Y_pred)
accuracy = float(cm.diagonal().sum())/len(Y_test)
print("\nAccuracy Of SVM For The Given Dataset : ", accuracy)
#Visualizing the classifier
#Before we visualize we might need to encode the classes 'apple' and 'orange' into
numericals.We can achieve that using the label encoder.
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
Y_train = le.fit_transform(Y_train)
#After encoding , fit the encoded data to the SVM
from sklearn.svm import SVC
classifier = SVC(kernel='rbf', random_state = 1)
classifier.fit(X_train,Y_train)
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
plt.figure(figsize = (7,7))
X_set, y_set = X_train, Y_train
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1, stop = X_set[:, 0].max() + 1,
Step = 0.01), np.arange(start = X_set[:, 1].min() - 1, stop = X_set[:, 1].max() + 1, Step =
0.01))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(),
X2.ravel()]).T).reshape(X1.shape), alpha = 0.75, cmap = ListedColormap(('black',
'white')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1], c = ListedColormap(('red',
'orange'))(i), label = j)
plt.title('Apples Vs Oranges')
plt.xlabel('Weight In Grams')
plt.ylabel('Size in cm')
plt.legend()
plt.show()
```

Result :

The above code was executed and verified successfully.

EXPT NO: 7.i. DATE:	Implement ensembling techniques (i) Averaging method

Aim:

To write a program for implementing Averaging ensembling technique using Python

Algorithm:

Step 1: Load the iris dataset using the sklearn.datasets module.
Step 2: Split the dataset into training and testing sets using the train_test_split function from sklearn.model_selection module.
Step 3: Select only the sepal data for both training and testing datasets.
Step 4: Create an instance of the BaggingClassifier class from the sklearn.ensemble module.
Step 5: Fit the BaggingClassifier instance to the training data.
Step 6: Calculate the score of the BaggingClassifier on the testing data.
Step 7: Create an instance of the KNeighborsClassifier class and fit it to the training data.
Step 8: Calculate the score of the KNeighborsClassifier on the testing data.
Step 9: Define the make_meshgrid function to create a meshgrid for plotting the decision boundaries.
Step 10: Define the plot_contours function to plot the decision boundaries.
Step 11: Get the sepal length and sepal width data from the iris dataset.
Step 12: Create the meshgrid for plotting the decision boundaries using the make_meshgrid function.
Step 13: Plot the decision boundaries using the plot_contours function and the BaggingClassifier instance.
Step 14: Plot the actual data points for the versicolor and virginica classes using the scatter function.
Step 15: Show the plot.

Program

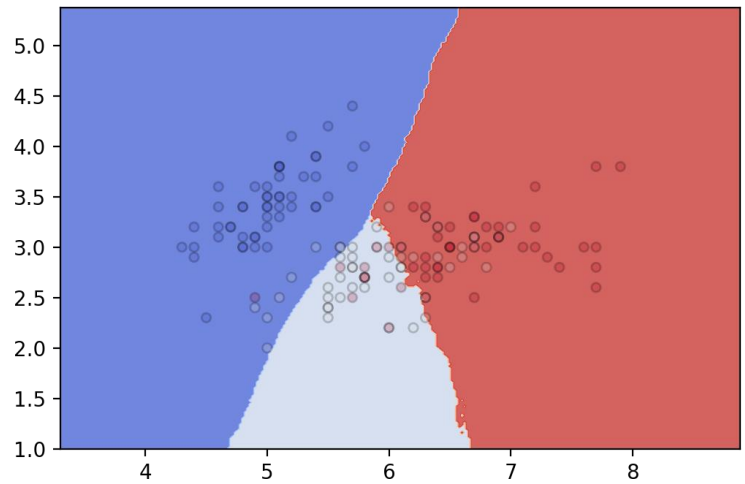
```
# Load the iris dataset
from sklearn import datasets
iris = datasets.load_iris()
# split into train and test datasets
from sklearn.model_selection import train_test_split
# just use the sepal data
X_train, X_test, y_train, y_test = train_test_split(iris.data[:,0:2],iris.target)
# Model the data set using Bagging Classifier
from sklearn.ensemble import BaggingClassifier
from sklearn.neighbors import KNeighborsClassifier
classifier = BaggingClassifier(base_estimator = KNeighborsClassifier(),
                               max_samples = 10,
                               n_estimators = 100)
classifier.fit(X_train,y_train)
```

Output

```
BaggingClassifier(base_estimator=KNeighborsClassifier(), max_samples=10,
```

```
            n_estimators=100)
# Calculate the score
classifier.score(X_test,y_test)
Output
0.7894736842105263
classifier_knn = KNeighborsClassifier()
classifier_knn.fit(X_train,y_train)
classifier_knn.score(X_test,y_test)
Output
0.7368421052631579
import numpy as np
def make_meshgrid(x, y, h=.02):
   x_min, x_max = x.min() - 1, x.max() + 1
   y_min, y_max = y.min() - 1, y.max() + 1
   xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                    np.arange(y_min, y_max, h))
   return xx, yy
def plot_contours(ax, clf, xx, yy, **params):
   Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
   Z = Z.reshape(xx.shape)
   out = ax.contourf(xx, yy, Z, **params)
   return out
X0, X1 = iris.data[:,0], iris.data[:, 1]
# Pass the data. make_meshgrid will automatically identify the min and max points to
draw the grid
xx, yy = make_meshgrid(X0, X1)
import matplotlib as mpl
import matplotlib.pyplot as plt
%matplotlib inline
mpl.rcParams['figure.dpi'] = 200
# plot the meshgrid
plot_contours(plt, classifier, xx, yy,
            cmap=plt.cm.coolwarm,
            alpha=0.8)
# plot the actual data points for versicolor and virginica
plt.scatter(X0, X1, c=iris.target,
            cmap=plt.cm.coolwarm,
            s=20, edgecolors='k',
            alpha=0.2)
```

Output



Here is a visual comparing ensemble of KNN classifier vs a single KNN classifier. You can see that the overfitting in the case of a single KNN classifier has been greatly reduced in the ensemble model.

Calculate the score

classifier.score(X_test,y_test)

Output

0.7894736842105263

classifier_knn.score(X_test,y_test)

Output

0.7368421052631579

The accuracy of the standalone KNN modeler seems to be better than the ensemble. That is because we have a small dataset. Once the model hits a large real world dataset, the ensemble performs much better – specifically with variance.

Result :

The above code was executed and verified successfully.

EXPT NO: 7.ii. DATE:	Implement Ensembling Techniques (ii) AdaBoost

Aim:

To write a program for implementing AdaBoost ensembling technique using Python

Algorithm:

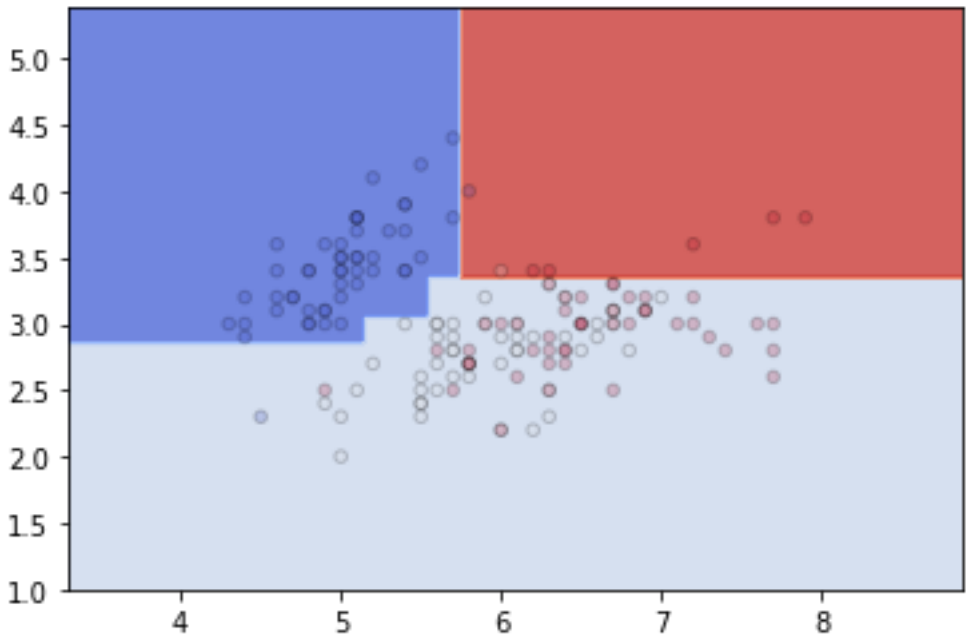
Step 1: Import the necessary libraries.
Step 2: Load the iris dataset.
Step 3: Split the dataset into training and testing sets.
Step 4: Create an instance of the AdaBoost classifier.
Step 5: Train the classifier on the training data.
Step 6: Create a meshgrid to plot the decision boundaries.
Step 7: Define a function to plot the decision boundaries and data points.
Step 8: Plot the decision boundaries and data points.
Step 9: Calculate the accuracy of the classifier on the test data.
Step 10: Output the accuracy score.

Program

```
from sklearn.ensemble import AdaBoostClassifier
classifier_adb =
AdaBoostClassifier(n_estimators=100,random_state=100,learning_rate=0.1)
classifier_adb.fit(X_train,y_train)
AdaBoostClassifier(learning_rate=0.1, n_estimators=100, random_state=100)
X0, X1 = iris.data[:,0], iris.data[:, 1]
# Pass the data. make_meshgrid will automatically identify the min and max points to
draw the grid
xx, yy = make_meshgrid(X0, X1)
import matplotlib as mpl
import matplotlib.pyplot as plt
%matplotlib inline
# plot the meshgrid
plot_contours(plt, classifier_adb, xx, yy,
            cmap=plt.cm.coolwarm,
            alpha=0.8)

# plot the actual data points for versicolor and virginica
plt.scatter(X0, X1, c=iris.target,
            cmap=plt.cm.coolwarm,
            s=20, edgecolors='k',
            alpha=0.2)
```

Output



classifier_adb.score(X_test,y_test)

Output

0.6578947368421053

The score of the classifier is almost similar to the Random Forest, but a bit higher than the regular decision tree score.

Result :

The above code was executed and verified successfully.

EXPT NO: 7.iii. DATE:	Implement Ensembling Techniques (iii) Gradient Descent

Aim:

To write a program for implementing Gradient Descent ensembling technique using Python

Algorithm:

Step 1: Import the necessary libraries.

Step 2: Load the Boston dataset using load_boston() from the datasets module.

Step 3: Split the dataset into training and testing sets using train_test_split method from the model_selection module.

Step 4: Instantiate a Gradient Boosting Regressor model from the ensemble module.

Step 5: Train the Gradient Boosting Regressor model on the training data using fit method

Step 6: Get the feature importance scores of the trained model using feature_importances_

Step 7: Evaluate the performance of the model on the test data using score() and calculate the R-squared value.

Step 8: Instantiate a Linear Regression model from the linear_model module and fit it to the training data.

Step 9: Evaluate the performance of the Linear Regression model on the test data using score method and calculate the R-squared value.

Gradient Boost is essentially a combination of Decision trees with some kind of Residuals minimizing algorithm (It could be Ordinary Least Squares or Gradient Descent). As long as you understand the concept of residuals and how to minimize them, we are good to go to understand Gradient boost.

So, by definition, Gradient Boost is a good for for regression problems. However, it can be adapted to Classification problems using the logit/expit function (used in Logistic Regression). So, let's get started with a regression problem, say the Boston Housting dataset.

```
from sklearn import datasets

boston = datasets.load_boston()

boston.feature_names
```

Output

```
array(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD',
```

```
'TAX', 'PTRATIO', 'B', 'LSTAT'], dtype='<U7')
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(boston.data[:,[0,4,5]],
boston.target, test_size=0.2, random_state=100)
from sklearn.ensemble import GradientBoostingRegressor
regressor = GradientBoostingRegressor(n_estimators = 100, max_depth = 4,
learning_rate = 0.05)
regressor.fit(X_train,y_train)
```

Output

```
GradientBoostingRegressor(learning_rate=0.05, max_depth=4)
regressor.feature_importances_
```

Output

```
array([0.1312375 , 0.20372316, 0.66503934])
```

```
# r2 score
regressor.score(X_test,y_test)
```

Output

```
0.806196207267125
```

```
from sklearn.linear_model import LinearRegression
regressor_lin = LinearRegression().fit(X_train,y_train)
regressor_lin.score(X_test,y_test)
```

Output

```
0.5941434559407395
```

Result :

The above code was executed and verified successfully.

EXPT NO: 8.i. DATE:	Implement Clustering Algorithms i) K-Means Clustering

Aim:

To implement K-Means clustering algorithm using Python

Algorithm:

Algorithm

Step 1:Import the necessary libraries: Import KMeans from sklearn.cluster and pandas for data manipulation.

Step 2:Read the data: Read the CSV file containing customer purchase data into a DataFrame called data.

Step 3:Initialize KMeans: Create a KMeans object with the desired number of clusters (2).

Step 4:Fit KMeans: Call the fit() method on the KMeans object to perform clustering.

Step 5:Predict cluster labels: Call the predict() method on the fitted KMeans object to assign data points to clusters.

Step 6:Get cluster labels: Store the predicted cluster labels in a variable called labels.

Step 7:Print cluster labels: Print the labels variable to display the cluster labels for each data point.

Step 8:Interpretation: Use the resulting cluster labels for further analysis or business decisions.

K-means is one of the most popular clustering algorithms, used to partition data into K clusters.

Sample Input:

Assume that we have a dataset of customer purchase records. The dataset has two features, namely 'Amount' and 'Frequency', which represent the amount spent and the frequency of visits to the store, respectively.

Program

```
from sklearn.cluster import KMeans
import pandas as pd
# Read the data
data = pd.read_csv(' D:\\KIOT\\AAAA\\AIML_LAB\\customer_purchases.csv')
# Initialize KMeans
kmeans = KMeans(n_clusters=2)
# Fit KMeans
kmeans.fit(data)
```

```
# Predict the cluster labels
labels = kmeans.predict(data)
# Print the cluster labels
print(labels)
```

Sample data set

Amount	Frequency
20	2
25	2
30	1
40	1
50	3
60	4
70	3
80	2
90	1

Output

[0 0 0 0 0 1 1 1 1]

Result :

The above code was executed and verified successfully.

EXPT NO: 8.ii. DATE:	Implement clustering algorithms ii) Hierarchical Clustering

Aim:

To implement Hierarchical clustering algorithm using Python

Algorithm:

Step 1:Import AgglomerativeClustering from sklearn.cluster and pandas for data manipulation.

Step 2:Read the CSV file containing animal data into a DataFrame called data.

Step 3:Initialize AgglomerativeClustering with the desired number of clusters (2).

Step 4:Fit AgglomerativeClustering by calling the fit() method on the AgglomerativeClustering object.

Step 5:Predict cluster labels using the labels_ attribute of the fitted AgglomerativeClustering object.

Step 6:Store the predicted cluster labels in a variable called labels.

Step 7:Print the labels variable to display the cluster labels for each data point.

Step 8:Interpretation: Use the resulting cluster labels for further analysis or business decisions.

Hierarchical clustering is another popular clustering algorithm that creates a tree-like structure of clusters.

Sample Input:

Assume that we have a dataset of animals, and we want to cluster them based on their attributes. The dataset has four features, namely 'Hair', 'Feathers', 'Eggs', and 'Milk', which represent whether the animal has hair, feathers, lays eggs, and produces milk, respectively.

Program

```
from sklearn.cluster import AgglomerativeClustering
import pandas as pd
# Read the data
data = pd.read_csv(' D:\\KIOT\\PRAVEEN\\AIML_LAB\\animals.csv')
# Initialize AgglomerativeClustering
hierarchical = AgglomerativeClustering(n_clusters=2)
# Fit AgglomerativeClustering
hierarchical.fit(data)
# Predict the cluster labels
labels = hierarchical.labels_
# Print the cluster labels
```

```
print(labels)
```

Sample data set

Hair	Feathers	Eggs	Milk
1	0	0	1
1	0	0	1
0	1	1	0
0	1	1	0
1	0	0	1
0	0	1	0

Output

[1 1 0 0 1 0]

The output shows that the algorithm has divided the animals into two clusters. The first cluster includes the animals that have hair and produce milk, while the second cluster includes the animals that have feathers and lay eggs.

Result :

The above code was executed and verified successfully.

EXPT NO: 8.iii. DATE:	Implement clustering algorithms iii) DBSCAN Clustering

Aim:

To implement DBSCAN clustering algorithm using Python

Algorithm:

Step 1: Import DBSCAN from sklearn.cluster and pandas for data manipulation.

Step 2: Read the CSV file containing customer purchase data into a DataFrame called data.

Step 3: Initialize DBSCAN with the desired hyperparameters: epsilon (eps) and minimum number of samples (min_samples).

Step 4: Fit DBSCAN by calling the fit() method on the DBSCAN object.

Step 5: Predict cluster labels using the labels_ attribute of the fitted DBSCAN object.

Step 6: Store the predicted cluster labels in a variable called labels.

Step 7: Print the labels variable to display the cluster labels for each data point.

Step 8: Interpretation: Use the resulting cluster labels for further analysis or business decisions.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a clustering algorithm that groups together points that are closely packed together, while marking points that are isolated as noise.

Sample Input:

Assume that we have a dataset of customers who visit a store, and we want to cluster them based on their shopping behavior. The dataset has two features, namely 'Amount' and 'Frequency', which represent the amount spent and the frequency of visits to the store, respectively.

Program

```
from sklearn.cluster import DBSCAN
import pandas as pd
# Read the data
data = pd.read_csv(' D:\\KIOT\\AAAA\\AIML_LAB\\customer_purchases.csv')
# Initialize DBSCAN
dbscan = DBSCAN(eps=15, min_samples=2)
# Fit DBSCAN
dbscan.fit(data)
# Predict the cluster labels
labels = dbscan.labels_
# Print the cluster labels
```

```
print(labels)
```

Sample data set:

Amount	Frequency
20	2
25	2
30	1
40	1
50	3
60	4
70	3
80	2
90	1

Output

[0 0 -1 -1 1 1 1 0 -1]

In this example, we set the eps parameter to 2 and the min_samples parameter to 2. These parameters control the density of the clusters and the minimum number of points required to form a cluster.

The output of the DBSCAN algorithm is an array of cluster labels, where -1 represents outliers.

Result :

The above code was executed and verified successfully.